

## Article

# Certificateless Provable Data Possession Scheme for Cloud-Based Electronic Health Records System

Yujie Wu <sup>1</sup>, Xiao Tan <sup>2</sup> and Qi Xie <sup>1,\*</sup> <sup>1</sup> School of Mathematics, Hangzhou Normal University, Hangzhou 311121, China; yujiewu@stu.hznu.edu.cn<sup>2</sup> School of Information Science and Technology, Hangzhou Normal University, Hangzhou 311121, China; xiaotan@hznu.edu.cn

\* Correspondence: qixie@hznu.edu.cn

**Abstract:** Electronic Health Records (EHRs: digital compilations of patient health status and diagnosis) are typically shared, analyzed, and stored on cloud servers. One operational challenge is to guarantee the accurate storage of EHRs, for instance, by utilizing Provable Data Possession (PDP). When a portion of one hospital's EHRs needs to be transferred to another, outsourcing the computational costs of data transfer to the cloud and ensuring the integrity of the data transferred off-site becomes a problematic issue. In this article, to tackle these two problems, we put forward a certificateless provable data possession scheme with outsourced data transmission on secure cloud storage. Our scheme achieves the following functions: ensuring the data integrity for the transferred data; only the data owner or the data recipient themselves can verify the integrity of their own remote data; delegating most of the computations to the public cloud server to enable data transferability. Finally, we analyze the security and efficiency of the concrete scheme. The analysis demonstrates that our scheme is demonstrably secure and efficient.

**Keywords:** cloud-based EHRs; provable data possession; certificateless; data integrity check; transferability; bilinear pairings

MSC: 94A60; 94A62



**Citation:** Wu, Y.; Tan, X.; Xie, Q. Certificateless Provable Data Possession Scheme for Cloud-Based Electronic Health Records System. *Mathematics* **2024**, *12*, 3883. <https://doi.org/10.3390/math12243883>

Academic Editor: Francesc Pozo

Received: 18 November 2024

Revised: 6 December 2024

Accepted: 8 December 2024

Published: 10 December 2024



**Copyright:** © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

## 1. Introduction

Electronic Health Records (EHRs) serve as a digital equivalent of a patient's paper chart, aiming to electronically document and oversee health and medical details. Their objective is to boost and optimize the efficacy, safety, and comprehensiveness of health data management. In modern healthcare settings, EHRs have progressively supplanted paper-based health records. Nevertheless, for an ordinary data holder, managing a substantial quantity of EHRs may prove infeasible, especially regarding storage and sharing with authorized entities. A cloud-based EHR system presents an alluring option as it permits EHRs to be uploaded and retained on cloud servers. A hindrance in the execution of Electronic Health Records (EHRs) is the necessity for EHRs systems of diverse healthcare establishments to exchange data and attain interoperability [1].

Amid the rapid expansion of data, it becomes an onerous task for users to maintain such a vast amount of data on local apparatuses. Consequently, an increasing number of organizations and individuals are inclined to entrust their data to the cloud for storage. However, the data stored in the cloud may encounter deterioration or loss due to the inescapable software glitches, hardware malfunctions, and human mistakes in the cloud milieu [2]. To ascertain whether the data are accurately stored in the cloud, a multitude of Provable Data Possession (PDP) schemes have been devised [3–7].

In the realm of remote data integrity auditing schemes, before uploading data blocks to the cloud, the data owner first needs to generate signatures for each of these individual data blocks. These signatures are meant to prove that the cloud actually has the relevant

data blocks during the integrity auditing process. After that, the data owner uploads both the data blocks and their associated signatures to the cloud. For example, from April to July 2023, anomalies occurred in Google Drive, which led to the loss of user data. Therefore, provable data possession (PDP) is a vital security technique for detecting both accidental losses of data integrity and malicious attacks on data integrity.

During the deployment of cloud-based Electronic Health Records (EHRs), data transmissions are unavoidable. For example, when patients convey their personal information to hospitals or when exchanges occur between diverse medical institutions.

In the domain of cloud computing, the data owner (DO) generally uploads a large quantity of data and assorted materials onto public cloud servers. Data transmission is a common operation in cloud services. The DO usually has the objective to transfer their cloud-based data to data recipients (DRs), as witnessed in situations such as business-to-business data acquisitions, the transfer of electronic medical records, and analogous cases. When portions of the DO's remote data are to be transferred to the DR, the DO's private key is kept confidential from the DR to safeguard the remaining data. Hence, these DO's data security mechanisms must be in accordance with the DR.

In the circumstance where the data owner's (DO's) data are downloaded to the data receiver (DR) and then processed and stored by the DR, considerable computational and communication resources would be expended. This method becomes impractical when the DR's processing capabilities are limited. In such situations, the computations must be delegated to the cloud server. Thus, transferable remote data integrity verification emerges as a crucial security technique, facilitating the confirmation of the integrity of the transferred data. Specifically, it aids in detecting both inadvertent integrity losses and malicious integrity assaults on the transferred data.

On the other hand, since the scale of the data reflects the privacy of the data owner (DO), it is of the highest significance that our proposed approach safeguards the privacy of the data volume. Only the DO or the data receiver (DR) itself should possess the capacity to evaluate the integrity of their own remote data. Private remote data integrity verification offers a practical and effective solution for this particular application.

To address the aforementioned issues, our study focuses on transferable and private remote data integrity verification in the public cloud context. This research encompasses the construction of the system model, the definition of the security model, the design of a specific scheme, as well as its associated security analysis and performance assessment.

## 2. Motivation and Contribution

Driven by the application requirements, we introduce a new provable data possession (PDP) scheme that accomplishes the following four functions:

1. Outsourcing a majority of computations to the public cloud server to achieve data transferability. This allows the data recipient to generate their own tags and verify the integrity of the data without having to download the entire dataset stored on the cloud.
2. Guaranteeing the data integrity of the transferred data. Converting the data owner's remote data into the data recipient's data, where the integrity can only be verified by the data recipient.
3. Our scheme is founded on certificateless cryptography, which streamlines the complicated certificate management process.
4. We conduct a security analysis of the proposed scheme and also demonstrate its performance through specific implementations. The results indicate that the proposed scheme attains satisfactory security and efficiency levels.

## 3. Related Work

Remote data integrity auditing mechanisms are prevalently employed in cloud storage environments to guarantee the integrity and security of remotely stored data. Often, a Third Party Auditor (TPA) is integrated into such mechanisms to alleviate the computational

strain on the user's side. Ateniese et al. [8] were the pioneers in introducing the concept of Provable Data Possession (PDP), with the objective of ensuring data possession on untrusted cloud servers. Their approach employs homomorphic authenticators and random sampling techniques to accomplish blockless verification and diminish I/O expenses. The primary goal is to verify that the cloud provider truly holds the data it professes to store, without the necessity of downloading the entire dataset.

Juels and Kaliski [9] devised a model known as Proof of Retrievability (POR) and presented a practical approach. This approach enables the retrieval of data stored in the cloud and ensures the integrity of the retrieved data. Ordinarily, it entails challenging the cloud server to demonstrate that it can retrieve specific portions of the data in a verifiable manner.

Shacham and Waters [10] proposed both private and public remote data integrity auditing strategies. These strategies incorporate techniques such as pseudorandom functions and BLS signatures to ensure data integrity while either maintaining privacy or permitting public verification of integrity, respectively. The private strategy may concentrate on ensuring that only authorized entities can validate the integrity of the data, whereas the public strategy allows anyone to do so.

Tan et al. [11] proposed a novel framework for POR and formalized this concept as Proof of Retrievability with Flexible Designated Verification (PoR-FDV). In their strategy, both private and public verifiability can be supported. These strategies endeavor to address the challenge of verifying the integrity of data stored in the cloud, where users may not have direct control over the infrastructure. They utilize cryptographic techniques and protocols to enable efficient and secure auditing without imposing excessively heavy computational burdens on users.

Wang et al. [12] introduced a privacy-protecting remote data integrity auditing strategy. This strategy employs random masking techniques to safeguard data privacy. Similarly, Worku et al. [13] utilized a distinct random masking technique to formulate a remote data integrity auditing strategy. This particular strategy exhibits enhanced efficiency in protecting data privacy compared to previous strategies such as the one proposed in [12]. To reduce the computational load associated with signature generation on the user's side, Guan et al. [14] designed a remote data integrity auditing strategy based on indistinguishability obfuscation techniques. Shen et al. [15] introduced a lightweight remote data integrity auditing strategy. This was achieved by incorporating a Third Party Medium (TPM) that assists users in generating signatures while also safeguarding data privacy.

Ateniese et al. [16] were the initial proponents of a partially dynamic PDP strategy. Subsequently, Erway et al. [17] utilized skip lists to construct a fully dynamic data auditing strategy. Wang et al. [18] put forward another remote data integrity auditing strategy. This strategy supports full data dynamics by leveraging Merkle Hash Trees. To mitigate the risks associated with user key exposure, Yu et al. [19,20], and Yu and Wang [21] proposed key-exposure-resistant remote data integrity auditing strategies. These strategies are founded on the key update technique described in [22].

Data sharing represents a crucial application within the context of cloud storage. Wang et al. [23] modified the ring signature to protect user identity privacy and concurrently ensure data integrity in shared cloud storage. Yang et al. [24] developed an efficient shared data integrity auditing strategy. This strategy not only safeguards user identity privacy but also enables the traceability of user identities. Fu et al. [25] designed a privacy-aware shared data integrity auditing strategy. This was accomplished by using verifiable group signatures to uphold both data integrity and user privacy. Wang et al. [26] proposed a shared data integrity auditing strategy with user revocation capabilities. The strategy utilizes proxy re-signature techniques to effectively manage user access changes. Luo et al.'s strategy [27] employed Shamir secret sharing to construct a shared data integrity auditing strategy. This strategy supports user revocation and facilitates efficient management of user access control while maintaining data integrity.

The aforementioned approaches rely on a Public Key Infrastructure (PKI), which gives rise to substantial overheads due to complex certificate management. To streamline this process, Wang [28] proposed an identity-based remote data integrity auditing strategy for multicloud storage. This strategy utilizes user identity information such as name or email instead of public keys. Wang et al. [29] introduced a novel identity-based proxy-oriented remote data integrity auditing strategy. In this strategy, a proxy is engaged to process data on behalf of users. Yu et al. [30] developed a remote data integrity auditing strategy. This strategy offers remarkable data privacy preservation within identity-based cryptosystems. Wang et al. [31] suggested an identity-based data integrity auditing strategy. This strategy ensures unconditional anonymity and incentive. Zhang et al. [32] proposed an identity-based remote data integrity auditing strategy. This strategy supports efficient user revocation in shared data scenarios.

Other aspects, such as privacy-preserving authenticators [33] and data deduplication [34] in remote data integrity auditing have also been investigated. However, to curtail computational and communication overheads, outsourcing data transmission is also a crucial technique. In this paper, we propose the first certificateless provable data possession strategy for secure cloud storage, which supports outsourced data transmission.

The remainder of this paper is structured as follows: In Section 4, we present notations and preliminaries. In Section 5, the system model and security model are introduced. We describe the proposed strategy in Section 6. In Section 7, the security analysis and the performance evaluation are provided. Finally, we conclude our paper in Section 8.

#### 4. Notions and Preliminaries

We present some notations that are employed in the descriptions of our scheme in Table 1.

**Table 1.** Notations.

| Notation                       | Meaning                                                                                |
|--------------------------------|----------------------------------------------------------------------------------------|
| $p$                            | One large prime                                                                        |
| $\mathbb{G}_1, \mathbb{G}_2$   | Multiplicative cyclic groups with order $p$                                            |
| $g$                            | A generator of group $\mathbb{G}_1$                                                    |
| $e$                            | A bilinear pairing map $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$ |
| $Z_p^*$                        | A prime field with nonzero elements                                                    |
| $n$                            | The number of data blocks of file $F$                                                  |
| $F = \{f_1, f_2, \dots, f_n\}$ | The original file $F$                                                                  |
| $ID_O, ID_C, ID_R$             | The identities of the data owner, the cloud and the data recipient                     |
| $msk$                          | The master secret key                                                                  |
| $sk_O, pk_O$                   | The data owner's private key and public key                                            |
| $sk_C, pk_C$                   | The cloud's private key and public key                                                 |
| $sk_R, pk_R$                   | The data recipient's private key and public key                                        |

In addition, some mathematical knowledge is also utilized in this paper, which encompasses the bilinear map and the Computational Diffie–Hellman (CDH) problem.

##### 1. Bilinear Map

Let  $G_1, G_2$  be two multiplicative cyclic groups of large prime order  $p$ , and  $g$  be a generator of  $G_1$ . The bilinear map is a map  $e : G_1 \times G_1 \rightarrow G_2$  possessing the following characteristics:

- Bilinearity: for all  $u, v \in G_1$  and  $a, b \in Z_p^*$ ,  $e(u^a, v^b) = e(u, v)^{ab}$ .
- Computability: there exists an efficiently computable algorithm for computing map  $e$ .

- Non-degeneracy:  $e(g, g) \neq 1$ .

## 2. Computational Diffie–Hellman(CDH) Problem

For unknown  $x, y \in \mathbb{Z}_p^*$ , given  $g, g^x$  and  $g^y$  as input, output  $g^{xy} \in G_1$ . The CDH assumption in  $G_1$  holds if it is computationally infeasible to solve the CDH problem in  $G_1$ .

## 5. System Model and Security Model

### 5.1. System Model

The system model involves five kinds of different entities: the cloud, the data owner (DO), the sanitizer, the Key Generate Center (KGC) and the data recipient (DR), as shown in Figure 1.

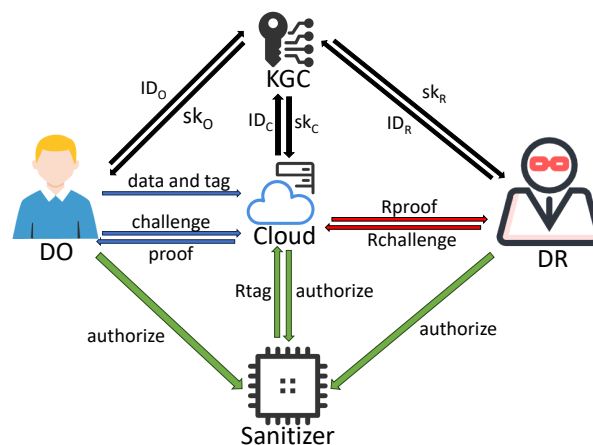


Figure 1. System model.

- **Cloud:** The cloud offers users vast amounts of data storage space along with substantial computational resources. By means of cloud storage services, users are able to upload their data to the cloud and share it with other individuals.
- **DO:** The DO stores a large volume of data on the cloud. Prior to the transfer of the data, the DO conducts the remote data integrity checking on their own. Once the data have been transferred, the transferred data integrity checking is carried out by the data recipient.
- **KGC:** The KGC is trusted by other entities. It undertakes the responsibility of generating the system public parameters as well as the private keys corresponding to the identity IDs of other entities.
- **DR:** The DR inherits the DO's remote data. Then, the DR can check the transferred data integrity by himself.
- **Sanitizer:** The sanitizer is a translation server used for proxy re-signing. The sanitizer transfers the DO's tag to the DR's Rtag with the help of the DR and the cloud. (For the transferred block, Rtag denotes the created tag for the DR)

The data owner, data recipient, and cloud severally complete the registration process at the key generation center to procure private keys and public keys. The data owner initially uploads its data to the cloud subsequent to generating tags, and at this juncture, the data owner can issue a challenge to the cloud to attain data integrity verification. When data transmission transpires, the sanitizer uploads the Rtag to the cloud after obtaining authorization, and then the data recipient can independently execute a challenge and obtain data integrity verification.

### 5.2. Design Goals

The proposed protocol is expected to meet the following requirements:

- **Correctness:**
  - **Private Key Correctness:** It is necessary to make sure that when the Private Key Generator (PKG) dispatches a correct private key to the data owner (DO), data recipient (DR), or the cloud, this private key can successfully undergo and pass the verification procedures carried out by the DO, DR, or the cloud, respectively.
  - **Signature Correctness:** It should be guaranteed that when the DO and the sanitizer send valid tags or Rtags to the cloud, these tags or Rtags are able to pass the verification process implemented by the cloud.
  - **Auditing Correctness:** One must ensure that when the cloud accurately stores the data belonging to the DO, the proof generated by the cloud can pass the verification processes of both the DO and the DR.
- **Auditing Soundness:** It is required to assure that if the cloud does not actually store the intact data of the DO, then it will not be able to pass the verification procedures conducted by either the DO or the DR.

### 5.3. Definition

**Definition:** The scheme consists of the following nine algorithms: *Setup*, *TagGen*, *ChalGen*, *ProofGen*, *ProofVerify*, *Authorized* and *RTagGen*, *RTagCheck*, *RProofGen*, and *RProofCheck*. Specifically, these algorithms are described as follows:

1. **Setup:** Taking a security  $k$  as input, the KGC generates the public keys  $pp$  and the respective public keys and private keys of the KGC, DO, cloud, and DR. Otherwise, the DO generates public values  $\{A_j = g^{a_j}\}_{j=1,2,\dots,s}$  used to generate tags and the KGC adds them into the system parameter after verifying.
2. **TagGen:** It is executed by the DO. Taking as input the parameter  $pp$ , the raw file  $F$  and private keys  $sk_{O1}$  and  $sk_{O2}$ , the algorithm outputs a tag set  $\mathcal{T}$  of  $F$ . Then, the cloud accepts it after verifying.
3. **ChalGen:** It is run by the DO, to generate a random challenge set  $chal = (c, k_1, k_2)$  and sends it to the cloud.
4. **ProofGen:** The cloud generates the corresponding proof  $P$  based on the challenge set  $chal$  and the data  $\mathcal{T}$  stored therein.
5. **ProofVerify:** The DO performs the algorithm to check whether the outsource  $F$  is intact. It takes the parameter  $pp$ , the private key  $sk_{O2}$ , the integrity proof  $P$ , and the challenge information  $chal$  as input, and judges whether the cloud stores the data correctly.
6. **Authorized and RTagGen:** It is an algorithm interacted between the DO, DR, cloud, and sanitizer. The sanitizer generates the R-tag  $\Sigma$  of the file blocks after obtaining the authorization from the other three entities.
7. **RTagCheck:** The cloud accepts the R-tag  $\Sigma$  after verifying its validity by using the public parameter  $pp$  and the private key  $sk_C$ .
8. **RProofGen:** It is executed by the cloud, to generate the response  $V$  including  $\{\hat{F}_i\}_{1 \leq i \leq s}$  and  $\Sigma_{proof}$  according to the challenge  $chal'$ .
9. **RProofCheck:** Taking as input the parameter, the response  $V$  and the private key  $sk_R$ , the DR verifies the integrity of the file  $F$  on the cloud.

#### 5.4. Security Model

##### 5.4.1. Informal Security

A PDP scheme ought to fulfill the following security requisites:

1. Internal security: This pertains to the security against untrustworthy sanitizers and the DR. It can be partitioned into two distinct security assurances.
  - Limited sanitizer security: The sanitizer is incapable of generating the block Rtag pairs for the DR without the authorization of either the DO, the DR, or the cloud. Even in the event of collusion between the sanitizer and the cloud, they cannot breach this limited sanitizer security.
  - DO security: Even if the sanitizer, DR, and the cloud collude, they can not forge the tag behalf of DO.
2. External security: It is the safeguard against foes external to the sanitizer Rtag framework (meaning they are distinct from the sanitizer, DO, and DR). The adversary is incapable of fabricating the block tag pairs or block Rtag pairs on behalf of the DO or DR. Notably, the cloud lacks the ability to forge the block tag pairs or block Rtag pairs on behalf of the DO or DR.
3. During the RGenProof phase, in the event that certain challenged block R-tag pairs have been either modified or deleted, the response provided by the malicious cloud can only pass the RCheckProof phase with a probability that is so small as to be negligible.

In the aforementioned security requirements, the collusion among the sanitizer, the cloud, and the DO is not taken into account. Given that the DO's remote data are transferred to the DR with the authorization of the DR, and this transfer occurs under the authorization and supervision of a trusted third party, it is challenging to facilitate the collusion of the sanitizer, the cloud, and the DO.

Additionally, to ensure the practicality of the Provable Data Possession (PDP) scheme, the scheme ought to exhibit efficiency in terms of both communication and computation overhead.

##### 5.4.2. Formal Security

For the sake of enhancing the practicality of the protocol, it ought to fulfill the subsequent security prerequisites. (1) All the data stored within the cloud should be subject to verification by the DO (data owner) or DR (data receiver). More precisely, in the event that both the cloud and the DO/DR execute the algorithms in an honest manner, the integrity of all the data will be accurately verified. (2) In case any portion of the data is either modified or lost, the cloud must not be able to pass the auditing verification. In other words, if data corruption occurs, the cloud is incapable of forging a valid proof or R-proof to get through the verification process.

To fulfill these security requirements, we take into account three categories of adversaries, specifically,  $\mathcal{A}_1$ ,  $\mathcal{A}_2$ , and  $\mathcal{A}_3$ .  $\mathcal{A}_1$  and  $\mathcal{A}_2$  make efforts to create forged tags or R-tags. More precisely,  $\mathcal{A}_1$  and  $\mathcal{A}_2$  have the ability to substitute the public keys of the data owner (DO) or the data recipient (DR). However, they do not possess access to the master secret key. On the contrary,  $\mathcal{A}_3$  attempts to fabricate the integrity proof or R-proof with the aim of passing the verification process. We define the security model by utilizing games that are played separately between the challenger  $\mathcal{C}$  and each of these three types of adversaries.

In an attempt to meet these security requirements, we consider three different kinds of adversaries, namely,  $\mathcal{A}_1$ ,  $\mathcal{A}_2$ , and  $\mathcal{A}_3$ .  $\mathcal{A}_1$  and  $\mathcal{A}_2$  endeavor to generate fake tags or R-tags. Specifically,  $\mathcal{A}_1$  and  $\mathcal{A}_2$  are capable of replacing the public keys of the DO/DR, yet they have no access to the master secret key. In contrast,  $\mathcal{A}_3$  tries to counterfeit the integrity proof or R-proof in order to pass the verification. We define the security model through games that are played between the challenger  $\mathcal{C}$  and each of the three types of adversaries, respectively.

Game 1: It is played between  $\mathcal{C}$  and  $\mathcal{A}_1$ .

**Setup Phase:** The challenger  $\mathcal{C}$  executes the Setup algorithm to obtain the system parameters  $pp$  and the master secret key. After that,  $\mathcal{C}$  forwards  $pp$  to  $\mathcal{A}_1$ .

**Query Phase:**  $\mathcal{A}_1$  initiates a series of queries, and  $\mathcal{C}$  replies correspondingly.

1. Hash Query:  $\mathcal{A}_1$  adaptively presents a sequence of hash queries to  $\mathcal{C}$ . In return,  $\mathcal{C}$  supplies the relevant hash values to  $\mathcal{A}_1$ .
2. Partial Private Key Query:  $\mathcal{A}_1$  adaptively picks  $ID_O$  and asks for the partial private key. Subsequently,  $\mathcal{C}$  sends the corresponding partial private key to  $\mathcal{A}_1$ .
3. Secret Key Query:  $\mathcal{A}_1$  adaptively opts for  $ID_O$  and requests the secret key. Then,  $\mathcal{C}$  sends the corresponding secret key to  $\mathcal{A}_1$ .
4. PublicKey Query:  $\mathcal{A}_1$  adaptively selects  $ID_O$  and enquires about the public key.  $\mathcal{C}$  then provides the corresponding public key to  $\mathcal{A}_1$ .
5. PublicKey Replace:  $\mathcal{A}_1$  has the capacity to replace the public key of  $ID$  with any random value.  $\mathcal{C}$  accordingly replaces the public key with the one chosen by  $\mathcal{A}_1$ .
6. Tag Query:  $\mathcal{A}_1$  adaptively chooses the tuple  $(f, ID)$  and sends it to  $\mathcal{C}$  so as to get the tag on block  $f$  from the data owner with the identity  $ID$ .  $\mathcal{C}$  runs the TagGen algorithm to generate the tag on block  $f$  and offers the corresponding tag value to  $\mathcal{A}_1$ .

**Forgery Phase:**  $\mathcal{A}_1$  generates a tag  $\sigma'$  for block  $b'$  with the identity  $ID'$ .  $\mathcal{A}_1$  is considered to win the game when the following conditions are fulfilled:

1. The forged tag  $\sigma'$  is valid for block  $b'$  with the identity  $ID'$  and the corresponding public key  $pk_{ID'}$ .
2.  $\mathcal{A}_1$  did not request the complete secret key of the user identified by the identity  $ID'$ .
3.  $\mathcal{A}_1$  did not ask for the partial secret key of the user identified by  $ID'$  and did not replace the public key identified by  $ID'$ .
4.  $\mathcal{A}_1$  did not query for the tag value corresponding to  $(b', ID')$ .

**Definition 1** (Unforgeability of Tags). Suppose the probability of adversary  $\mathcal{A}_1$  winning Game 1 is  $\epsilon$ , and the probability of constructing an attacker that can solve the CDH problem is  $\epsilon'$ . If  $\epsilon \leq p(n) \cdot \epsilon'$  for any polynomial  $p(n)$ , the probability  $\epsilon$  of adversary  $\mathcal{A}_1$  winning Game 1 is negligible, meaning that the tag of each block is unforgeable.

Game 2: This game takes place between  $\mathcal{C}$  and  $\mathcal{A}_2$ .

**Setup Phase:** The challenger  $\mathcal{C}$  executes the Setup algorithm to acquire the system parameters  $pp$  and the master secret key. Next,  $\mathcal{C}$  transmits  $pp$  to  $\mathcal{A}_2$ .

**Query Phase:**  $\mathcal{A}_2$  raises a series of queries, and  $\mathcal{C}$  replies to them one-by-one.

1. Hash Query: The adversary  $\mathcal{A}_2$  adaptively presents a sequence of hash queries to the challenger  $\mathcal{C}$ . In response,  $\mathcal{C}$  provides the corresponding hash values to  $\mathcal{A}_2$ .
2. Partial Private Key Query:  $\mathcal{A}_2$  adaptively picks  $ID_O$  and requests the partial private key. Thereafter,  $\mathcal{C}$  sends the corresponding partial private key to  $\mathcal{A}_2$ .
3. Secret Key Query:  $\mathcal{A}_2$  adaptively chooses  $ID_O$  and enquires about the secret key. Subsequently,  $\mathcal{C}$  sends the corresponding secret key to  $\mathcal{A}_2$ .
4. TableProQuery:  $\mathcal{A}_2$  adaptively selects  $ID_O$  and  $ID_R$  and makes a query about the table  $T_{pro}$ . Then,  $\mathcal{C}$  sends the corresponding partial R-tag to  $\mathcal{A}_2$ .
5. PublicKey Query:  $\mathcal{A}_2$  adaptively selects  $ID_O$  and asks for the public key.  $\mathcal{C}$  then supplies the corresponding public key to  $\mathcal{A}_2$ .
6. PublicKey Replace:  $\mathcal{A}_2$  has the capability to substitute the public key of  $ID$  with any value.  $\mathcal{C}$  accordingly replaces the public key with the one selected by  $\mathcal{A}_2$ .
7. Tag Query:  $\mathcal{A}_2$  adaptively opts for the tuple  $(f, ID)$  and sends it to  $\mathcal{C}$  aiming to obtain the tag on block  $f$  from the data owner with the identity  $ID$ .  $\mathcal{C}$  runs the TagGen algorithm to generate the tag on block  $f$  and gives the corresponding tag value to  $\mathcal{A}_2$ .

**Forgery Phase:**  $\mathcal{A}_2$  generates a tag  $\sigma'$  for block  $b'$  with the identity  $ID'$ .  $\mathcal{A}_2$  is deemed to win the game when the following conditions are met:

1. The forged tag  $\sigma'$  is valid for block  $b'$  with the identity  $ID'$  and the public key  $pk_{ID'}$ .
2.  $\mathcal{A}_2$  did not query the entire secret key of the user identified by the identity  $ID'$ .
3.  $\mathcal{A}_2$  did not request the partial secret key of the user identified by  $ID'$  and did not replace the public key identified by  $ID'$ .
4.  $\mathcal{A}_2$  did not query the tag value corresponding to  $(b', ID')$ .

**Definition 2** (Unforgeability of Rtags). Suppose the probability of adversary  $\mathcal{A}_2$  winning Game 2 is  $\epsilon$ , and the probability of constructing an attacker that can solve the CDH problem is  $\epsilon'$ . If  $\epsilon \leq p(n) \cdot \epsilon'$  for any polynomial  $p(n)$ , the probability  $\epsilon$  of adversary  $\mathcal{A}_2$  winning Game 2 is negligible, indicating that the Rtag of each block is unforgeable.

Game 3: This game unfolds between the challenger  $\mathcal{C}$  and the adversary  $\mathcal{A}_3$ . In this scenario,  $\mathcal{A}_3$  is regarded as the untrustworthy cloud. The key aspect of Game 3 lies in determining whether the cloud can fabricate the auditing proof without having access to the precise data. The specific steps of the game are detailed below.

**Setup Phase:** The challenger  $\mathcal{C}$  executes the  $Setup(1^k)$  procedure to obtain the master secret key ( $msk$ ), the public parameters ( $pp$ ), and the private key of the data owner. Subsequently,  $\mathcal{C}$  keeps the master secret key ( $msk$ ) and the private key of the data owner confidential and forwards  $pp$  to  $\mathcal{A}_3$ .

**Challenge Phase:**  $\mathcal{C}$  presents a random challenge ( $chal$ ) to  $\mathcal{A}_3$  and requires  $\mathcal{A}_3$  to respond with the corresponding proof.

**Forge Phase:**  $\mathcal{A}_3$  creates the proof ( $P$ ) based on  $chal$ . If  $P$  can successfully pass the verification process carried out by  $\mathcal{C}$ , it will be considered that  $\mathcal{A}_3$  wins Game 3.

**Definition 3** (Unforgeability of proof). If the probability  $\epsilon$  that any Probabilistic Polynomial Time adversary  $\mathcal{A}_3$  triumphs in Game 3 is negligible, that is to say, there exists a sufficiently large number  $p$  such that  $\epsilon \leq \frac{1}{p}$ , then the proof furnished by the cloud for each challenge is impervious to forgery.

### 5.5. The Detectability

Assume that the original file  $F$  stored in the cloud is divided into  $n$  blocks. Out of these blocks, the cloud modifies or deletes  $k$  data blocks within file  $F$ . Meanwhile, the data owner challenges  $c$  data blocks. If our remote data integrity auditing scheme is  $\left(\frac{k}{n}, 1 - \left(\frac{n-k}{n}\right)^c\right)$  detectable, it implies that when the ratio of the modified blocks within the file is  $\frac{k}{n}$ , the probability of detecting those modified blocks is no less than  $1 - \left(\frac{n-k}{n}\right)^c$ .

## 6. The Proposed Scheme

### 6.1. Setup

This algorithm is implemented by the Key Generation Center (KGC) along with other entities with the aim of generating the system parameters and private keys.

1. In this phase, the KGC generates the system parameters via the following steps:
  - Given a security parameter  $1^k$ , the cloud selects two cyclic multiplicative groups  $\mathbb{G}_1$  and  $\mathbb{G}_2$  having a large prime order  $q$ . Let  $g$  be a generator of  $\mathbb{G}_1$ . Let  $e$  denote a computable bilinear pairing:  $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$ .
  - The KGC randomly picks  $msk = x \in \mathbb{Z}_q^*$  as the master secret key and computes the master public key  $mpk = g^x$ .
  - The KGC chooses four distinct hash functions  $H_1 : \{0, 1\}^* \rightarrow \mathbb{G}_1$ ,  $H_2 : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$ ,  $H_3 : \mathbb{G}_1 \rightarrow \mathbb{Z}_q^*$ ,  $H_4 : \{0, 1\}^* \times \mathbb{Z}_q^* \rightarrow \mathbb{G}_1$ , a pseudo-random permutation  $\pi : \mathbb{Z}_q^* \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ , a pseudo-random function  $\phi : \mathbb{Z}_q^* \times \mathbb{Z}_q^* \rightarrow \mathbb{Z}_q^*$ , a

trapdoor function  $\omega : Z_q^* \times \{0,1\}^* \rightarrow \{0,1\}^*$ . Examples of applicable hash functions include SHA-256.

- The KGC publishes the system parameters  
 $pp = (q, g, \mathbb{G}_1, \mathbb{G}_2, e, mpk, H_1, H_2, H_3, H_4, \phi, \pi, \omega)$ .
- 2. The KGC generates the data owner's (DO's) partial private key in the following way:
  - The DO sends its identity  $ID_O \in \{0,1\}^*$  to the KGC to obtain its partial key.
  - The KGC computes  $ssk_O = H_1(ID_O)^x$  and returns  $ssk_O$  to the DO.
  - After receiving  $ssk_O$ , the DO verifies the correctness of the partial key by checking whether  $e(ssk_O, g) = e(H_1(ID_O), mpk)$ . If the verification fails, the DO rejects the partial private key and makes another request.
- 3. The DO generates its secret value and public key as follows:
  - The DO randomly selects  $x_O \in Z_q^*$ .
  - The DO computes  $sk_{O2} = (x_O + H_2(ssk_O))$  as its secret key.  
 In this case, the public key and secret key are  
 $sk_O = \{sk_{O1} = ssk_O, sk_{O2} = (x_O + H_2(ssk_O))\}, pk_O = g^{sk_{O2}}$ .
- 4. The cloud and the data receiver (DR) generate their public keys and secret keys

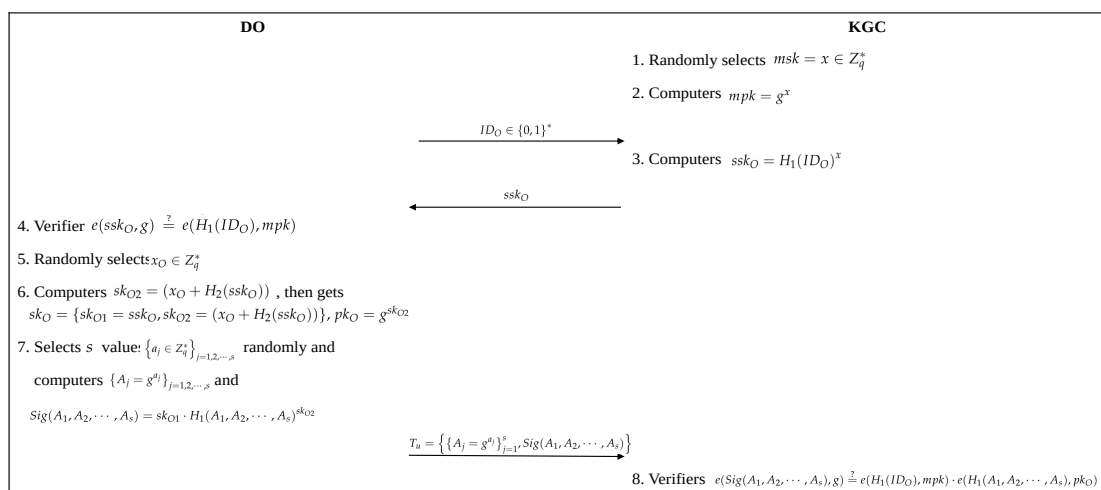
$$\{sk_C = x_C + H_2(H_1(ID_C)^x), pk_C = g^{sk_C}\}, \{sk_R = x_R + H_2(H_1(ID_R)^x), pk_R = g^{sk_R}\}$$

respectively, in a manner similar to the above steps, with one distinction.

- 5. To reduce communication and computation costs, the DO randomly selects  $s$  values  $\{a_j \in Z_q^*\}_{j=1,2,\dots,s}$  and computes  $s$  public values  $\{A_j = g^{a_j}\}_{j=1,2,\dots,s}$ .
  - The DO keeps  $\{a_j \in Z_q^*\}_{j \in [1,2,\dots,n]}$  confidential and sends  
 $T_u = \left\{ \{A_j = g^{a_j}\}_{j=1}^s, \text{Sig}(A_1, A_2, \dots, A_s) \right\}$  to the KGC, where  $\text{Sig}(A_1, A_2, \dots, A_s) = sk_{O1} \cdot H_1(A_1, A_2, \dots, A_s)^{sk_{O2}}$ .
  - The KGC verifies the correctness of the signature by

$$e(\text{Sig}(A_1, A_2, \dots, A_s), g) \stackrel{?}{=} e(H_1(ID_O), mpk) \cdot e(H_1(A_1, A_2, \dots, A_s), pk_O)$$

If the verification is successful, the KGC incorporates  $T_u$  into the system parameters. Otherwise, the KGC rejects it and notifies the DO to resend the public parameters. The interaction between the DO and the KGC during the *Setup* process is summarized in Figure 2.



**Figure 2.** The interaction between the DO and KGC in the *Setup*.

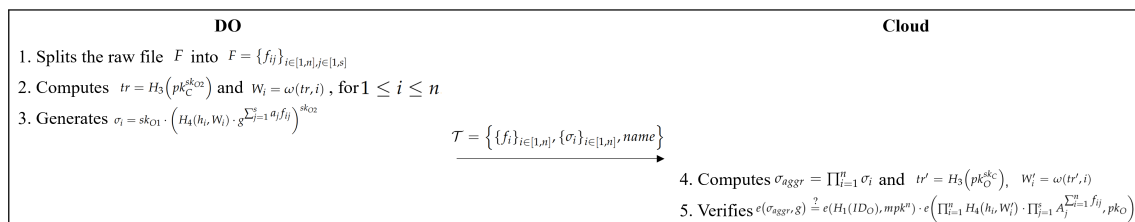
### 6.2. TagGen

This algorithm is implemented by the data owner (DO) and the cloud. Assume that the original file is denoted as  $F$ , and each file is assigned a unique identifier named  $name$ . The functioning of this algorithm is detailed below.

1. The DO divides the raw file  $F$  into  $n$  blocks, that is,  $F = \{f_1, f_2, \dots, f_n\}$ .
2. Each block  $f_i$  is further broken down into  $s$  sectors  $\{f_{ij}\}_{j \in [1,s]}$ , where  $f_{ij} \in Z_q^*$ . Consequently, the entire file can be expressed as  $F = \{f_{ij}\}_{i \in [1,n], j \in [1,s]}$ .
3. The DO computes  $tr = H_3(pk_C^{sk_{O2}})$  and  $W_i = \omega(tr, i)$ , for  $1 \leq i \leq n$ .
4. The DO generates the tag for  $f_i$  by computing  $\sigma_i = sk_{O1} \cdot \left( H_4(h_i, W_i) \cdot g^{\sum_{j=1}^s a_j f_{ij}} \right)^{sk_{O2}}$ , where  $h_i = name \| ID_O \| i, i \in [1, n]$ .
5. The DO sends  $\mathcal{T}$  to the cloud such that  $\mathcal{T} = \{\{f_i\}_{i \in [1,n]}, \{\sigma_i\}_{i \in [1,n]}, name\}$ . Finally, the DO deletes the original data except for  $\{sk_O, name\}$ .
6. The cloud verifies the information in the following manner. It generates the aggregated tag  $\sigma_{aggr} = \prod_{i=1}^n \sigma_i$  and computes  $tr' = H_3(pk_O^{sk_C})$ ,  $W'_i = \omega(tr', i)$ . Then, it verifies the following equation:

$$e(\sigma_{aggr}, g) = e(H_1(ID_O), mpk^n) \cdot e\left(\prod_{i=1}^n H_4(h_i, W'_i) \cdot \prod_{j=1}^s A_j^{\sum_{i=1}^n f_{ij}}, pk_O\right) \quad (1)$$

where  $h_i = name \| ID_O \| i, i \in [1, n]$ . If the equation does not hold, the cloud rejects and declines to store the DO's data. The process of *TagGen* is summarized in Figure 3.



**Figure 3.** The process of *TagGen*.

### 6.3. ChalGen

In order to verify the integrity of the entire file, DO randomly picks two values, namely,  $k_1$  and  $k_2$ , where both belong to  $Z_q^*$ . Here,  $k_1$  serves as a random seed for the pseudo-random permutation, while  $k_2$  acts as a random seed for the pseudo-random function, respectively. Along with  $c$ , which represents the number of challenge block numbers that need to be inspected, the DO sends  $chal = (c, k_1, k_2)$  to the cloud.

### 6.4. ProofGen

Upon receiving  $chal = (c, k_1, k_2)$  from DO, the cloud generates its proof as follows:

1. It generates the challenge set  $C = \{(v_\beta, c_\beta)\}$ , where  $v_\beta = \pi(k_1, \beta)$ ,  $c_\beta = \phi(k_2, \beta)$  for  $\beta \in [1, c]$ .
2. It computes  $\sigma_{proof} = \prod_{\beta=1}^c \sigma_{v_\beta}^{c_\beta}$  and  $M_j = \sum_{\beta=1}^c c_\beta f_{v_\beta j}$ , where  $j \in [1, s]$ .
3. The cloud returns the proof  $P = \{\sigma_{proof}, \{M_j\}_{j \in [1,s]}\}$  to the DO.

### 6.5. ProofVerify

Upon receiving proof  $P$  from the cloud, the DO verifies the proof  $P$  as follows:

1. The DO generates the challenge set  $C = \{(v_\beta, c_\beta)\}$ , where  $v_\beta = \pi(k_1, \beta)$ ,  $c_\beta = \phi(k_2, \beta)$  for  $\beta \in [1, c]$ .
2. The DO computes  $tr = H_3(pk_C^{sk_{O2}})$  and  $W_{v_\beta} = \omega(tr, v_\beta)$ , for  $1 \leq \beta \leq c$ .
3. The DO checks whether the following equation holds:

$$e(\sigma_{proof}, g) = e\left(H_1(ID_O)^{\sum_{\beta=1}^c c_\beta}, mpk\right) \cdot e\left(\prod_{\beta=1}^c H_4(h_{v_\beta}, W_{v_\beta})^{c_\beta} \cdot \prod_{j=1}^s A_j^{M_j}, pk_O\right) \quad (2)$$

where  $h_{v_\beta} = name \| ID_O \| v_\beta$ ,  $\beta \in [1, c]$ . If the equation holds, that means the cloud has stored the data correctly.

The above is the provable data possession scheme in the cloud storage server for the DO. When the DO's data need to be transferred to the DR, he can verify whether the cloud has stored the complete data of the DO through the following scheme.

### 6.6. Authorized and RTagGen

The DR's R-tag pairs  $\Sigma$  is generated by the following processes.

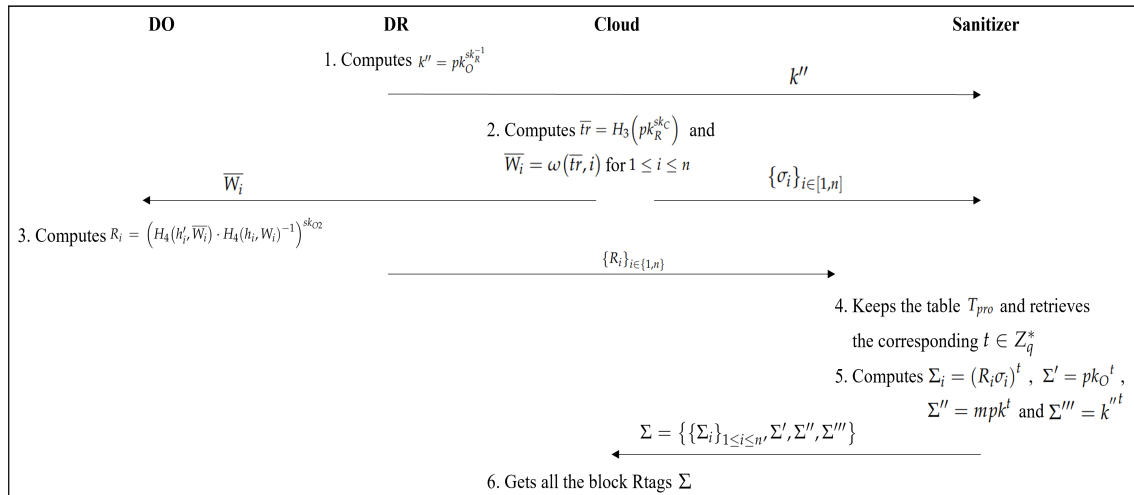
1. The DR computes the re-key  $k'' = pk_O^{sk_R^{-1}}$  and sends it to the sanitizer, which can be used to transfer the data owner's block tag pairs into the data steward's block R-tag pairs.
2. For  $1 \leq i \leq n$ , the cloud computes  $\bar{tr} = H_3(pk_R^{sk_C})$  and  $\bar{W}_i = \omega(\bar{tr}, i)$ . Then, it sends  $\bar{W}_i$  to the DO and sends  $\{\sigma_i\}_{i \in [1, n]}$  to the sanitizer.
3. The DO computes  $R_i = \left(H_4(h'_i, \bar{W}_i) \cdot H_4(h_i, W_i)^{-1}\right)^{sk_{O2}}$ , where  $h'_i = name \| ID_R \| i$ ,  $h_i = name \| ID_O \| i$ ,  $1 \leq i \leq n$ . Then, it sends  $\{R_i\}_{i \in [1, n]}$  to the sanitizer.
4. The table  $T_{pro}$  is initially empty and is kept by the sanitizer. If there exists the tuple (DO, \*) in the table  $T_{pro}$ , the sanitizer retrieves the tuple (data owner,  $t$ ) and gets  $t$ . Otherwise, the sanitizer chooses  $t \in Z_q^*$  and stores (data owner,  $t$ ) into the table  $T_{pro}$ .
5. Using the chosen or retrieved value  $t$ , the sanitizer computes the R-tag  $\Sigma$  as the following procedure.

$$\begin{aligned} \Sigma_i &= (R_i \sigma_i)^t; \\ \Sigma' &= pk_O^t; \\ \Sigma'' &= mpk^t; \\ \Sigma''' &= k''^t. \end{aligned}$$

6. It is noted that:

$$\begin{aligned} \Sigma_i &= sk_{O1}^t \cdot \left( H_4(h'_i, \bar{W}_i) \prod_{j=1}^s A_j^{f_{ij}} \right)^{t \cdot sk_{O2}}; \\ \Sigma' &= g^{sk_{O2} \cdot t}; \\ \Sigma'' &= g^{x \cdot t}; \\ \Sigma''' &= k''^t = g^{\frac{t \cdot sk_{O2}}{-sk_R}}. \end{aligned}$$

7. We denote the R-tag as  $\Sigma = \{\{\Sigma_i\}_{1 \leq i \leq n}, \Sigma', \Sigma'', \Sigma'''\}$ . Finally, the cloud gets all the block Rtags  $\Sigma$ . The processes of *Authorized and RTagGen* are summarized in Figure 4.



**Figure 4.** The processes of *AuthorizedandRTagGen*.

#### 6.7. *RTagCheck*

Given  $\{(f_i, \Sigma), 1 \leq i \leq n\}$ , where  $\Sigma = \{\{\Sigma_i\}_{1 \leq i \leq n}, \Sigma', \Sigma'', \Sigma'''\}$ , the cloud performs the procedures for every  $i, 1 \leq i \leq n$  below:

1. Computers  $\bar{tr} = H_3(pk_R^{sk_C})$  and  $\bar{W}_i = \omega(\bar{tr}, i)$ .
2. The cloud tests whether the following formulas hold:

$$e(\Sigma_i, g) \stackrel{?}{=} e\left(H_4(h'_i, \bar{W}_i) \prod_{j=1}^s A_j^{f_{ij}}, \Sigma'\right) \cdot e(H_1(ID_O), \Sigma'');$$

$$e(\Sigma', mpk) \stackrel{?}{=} e(pk_O, \Sigma'');$$

$$e(\Sigma', g) \stackrel{?}{=} e(pk_R, \Sigma''');$$

where  $h'_i = \text{name} \| ID_R \| i$ . If they hold, then the cloud accepts it. Otherwise, the block R-tag pair is rejected.

#### 6.8. *RProofGen*

Let the file block set be  $F = (f_1, f_2, \dots, f_n)$  and the corresponding R-tag set be  $\Sigma = \{\{\Sigma_i\}_{1 \leq i \leq n}, \Sigma', \Sigma'', \Sigma'''\}$ . Let the challenge be  $chal' = (c', k'_1, k'_2)$  where  $1 \leq c' \leq n, k_1, k_2 \in Z_q^*$ . The cloud performs the procedure below:

- For  $1 \leq \alpha \leq c'$ , computes  $v'_\alpha = \pi(k'_1, \alpha)$ ,  $c'_\alpha = \phi(k'_2, \alpha)$ .
- Computes  $\Sigma_0 = \prod_{\alpha=1}^{c'} \Sigma_{v'_\alpha}^{c'_\alpha}$ ,  $M'_j = \sum_{\alpha=1}^{c'} c'_\alpha f_{v'_\alpha j}$  for  $1 \leq j \leq s$ .
- Outputs  $V = \left(\{M'_j\}_{1 \leq j \leq s}, \Sigma_{proof} = (\Sigma_0, \Sigma', \Sigma'', \Sigma''')\right)$ .

#### 6.9. *RProofCheck*

For the challenge  $chal'$  and the response  $V$ , DR performs the procedures below:

- For  $1 \leq \alpha \leq c'$ , computes  $v'_\alpha = \pi(k'_1, \alpha)$ ,  $c'_\alpha = \phi(k'_2, \alpha)$ .
- Computes  $\hat{tr} = H_3(pk_C^{sk_R})$ ,  $\hat{W}_{v'_\alpha} = \omega(\hat{tr}, v'_\alpha)$ ,  $1 \leq \alpha \leq c'$ .
- Checks whether the following formulas hold:

$$\begin{aligned}
e(\Sigma_0, g) &\stackrel{?}{=} e\left(\prod_{\alpha=1}^{c'} \left(H_4(h'_{v'_\alpha}, \hat{W}_{v'_\alpha})\right)^{c'_\alpha} \cdot \prod_{j=1}^s A_j^{M'_j}, \Sigma'\right) \cdot e\left(H_1(ID_O) \Sigma_{\alpha=1}^{c'} c'_\alpha, \Sigma''\right) \\
e(\Sigma', mpk) &\stackrel{?}{=} e(pk_O, \Sigma''); \\
e(\Sigma', g) &\stackrel{?}{=} e(pk_R, \Sigma''');
\end{aligned} \tag{3}$$

If they hold, then the DR outputs “*success*”. Otherwise, the DR outputs “*failure*”.

## 7. Analysis of Our Scheme

We will carry out an analysis of the proposed scheme in terms of three aspects: its correctness, security, and performance evaluation.

### 7.1. Correctness

If the entities involved execute the operations in accordance with our scheme in an honest manner, then the Tags and Proofs will be able to pass their respective verification algorithms. The following Equations (4)–(6) prove Equations (1), (2), and (3), respectively.

$$\begin{aligned}
e(\sigma_{aggr}, g) &= e\left(\prod_{i=1}^j \sigma_i, g\right) \\
&= e\left(\prod_{i=1}^n sk_{O1} \cdot \left(H_4(h_i, W'_i) \cdot g^{\sum_{j=1}^s a_j f_{ij}}\right)^{sk_{O2}}, g\right) \\
&= e\left(\prod_{i=1}^n H_1(ID_O)^x \cdot \left(H_4(h_i, W_i) \cdot \prod_{j=1}^s A_j^{f_{ij}}\right)^{sk_{O2}}, g\right) \\
&= e(H_1(ID_O)^{nx}, g) \cdot e\left(\prod_{i=1}^n \left(H_4(h_i, W_i) \cdot \prod_{j=1}^s A_j^{f_{ij}}\right)^{sk_{O2}}, g\right) \\
&= e(H_1(ID_O), mpk^n) \cdot e\left(\prod_{i=1}^n H_4(h_i, W_i) \cdot \prod_{j=1}^s A_j^{\sum_{i=1}^n f_{ij}}, pk_O\right)
\end{aligned} \tag{4}$$

$$\begin{aligned}
e(\sigma_{proof}, g) &= e(\prod_{\beta=1}^c \sigma_{v_\beta}^{c_\beta}, g) \\
&= e(\prod_{\beta=1}^c \left(sk_{O1} \cdot \left(H_4(h_{v_\beta}, W_{v_\beta}) \cdot g^{\sum_{j=1}^s a_j f_{v_\beta j}}\right)^{sk_{O2}}\right)^{c_\beta}, g) \\
&= e(H_1(ID_O)^{x \cdot \sum_{\beta=1}^c c_\beta} \cdot \left(\prod_{\beta=1}^c H_4(h_{v_\beta}, W_{v_\beta}) \cdot g^{\sum_{j=1}^s a_j f_{v_\beta j}}\right)^{c_\beta sk_{O2}}, g) \\
&= e\left(\left(H_1(ID_O)^{\sum_{\beta=1}^c c_\beta}\right)^x, g\right) \\
&\quad \cdot e\left(\left(\prod_{\beta=1}^c H_4(h_{v_\beta}, W_{v_\beta})^{c_\beta} \cdot \prod_{j=1}^s A_j^{\sum_{\beta=1}^c c_\beta f_{v_\beta j}}\right)^{sk_{O2}}, g\right) \\
&= e\left(H_1(ID_O)^{\sum_{\beta=1}^c c_\beta}, mpk\right) \cdot e\left(\prod_{\beta=1}^c H_4(h_{v_\beta}, W_{v_\beta})^{c_\beta} \cdot \prod_{j=1}^s A_j^{M_j}, pk_O\right)
\end{aligned} \tag{5}$$

$$\begin{aligned}
e(\Sigma_0, g) &= e\left(\prod_{\alpha=1}^{c'_\alpha} \Sigma_{v'_\alpha}^{c'_\alpha}, g\right) \\
&= e\left(\prod_{\alpha=1}^{c'_\alpha} \left( sk_{O1}^t \cdot \left( H_4(h'_{v'_\alpha}, \overline{W}_{v'_\alpha}) \prod_{j=1}^s A_j^{f_{v'_\alpha j}} \right)^{t \cdot sk_{O2}} \right)^{c'_\alpha}, g\right) \\
&= e\left(\prod_{\alpha=1}^{c'_\alpha} \left( H_4(h'_{v'_\alpha}, \overline{W}_{v'_\alpha}) \prod_{j=1}^s A_j^{f_{v'_\alpha j}} \right)^{c'_\alpha}, g^{t \cdot sk_{O2}}\right) \cdot e\left(\prod_{\alpha=1}^{c'_\alpha} sk_{O1}^{t \cdot c'_\alpha}, g\right) \\
&= e\left(\prod_{\alpha=1}^{c'_\alpha} \left( H_4(h'_{v'_\alpha}, \hat{W}_{v'_\alpha}) \right)^{c'_\alpha} \cdot \prod_{j=1}^s \left( \prod_{\alpha=1}^{c'_\alpha} A_j^{f_{v'_\alpha j} \cdot c'_\alpha} \right), \Sigma'\right) \cdot e\left(H_1(ID_O) \sum_{\alpha=1}^{c'_\alpha} c'_\alpha, g^{x \cdot t}\right) \\
&= e\left(\prod_{\alpha=1}^{c'_\alpha} \left( H_4(h'_{v'_\alpha}, \hat{W}_{v'_\alpha}) \right)^{c'_\alpha} \cdot \prod_{j=1}^s A_j^{M'_j}, \Sigma'\right) \cdot e\left(H_1(ID_O) \sum_{\alpha=1}^{c'_\alpha} c'_\alpha, \Sigma''\right)
\end{aligned} \tag{6}$$

## 7.2. Security Analysis

**Theorem 1** (Unforgeability of tags). *If the probability of an attacker  $\mathcal{A}_1$  winning the game 1 is negligible, then our scheme satisfies tag unforgeability under Definition 1.*

**Proof.** In the first case, given an instance  $(g, \mathbb{G}, g^\alpha, g^\beta)$ , if the probability that  $\mathcal{A}_1$  wins the game 1 is not negligible, there exists a simulator  $S$  who can calculate  $g^{\alpha\beta}$  by using the power of  $\mathcal{A}_1$ . That is,  $S$  can solve the CDH problem.  $S$  simulates the interaction with  $\mathcal{A}_1$  as follows.

**Setup:** The entity  $S$  executes the Setup algorithm to generate the public parameters  $pp$ . It sets  $mpk = g^\alpha$  and randomly selects  $ID_O$  as a challenge identity. Subsequently,  $S$  randomly picks  $s$  random values  $(a_1, a_2, \dots, a_s) \in Z_q^*$  and computes  $A_k = g^{a_k}$  where  $k \in [1, s]$ .  $S$  keeps  $\alpha$  confidential and sends  $pp$  along with  $\{A_k\}_{k \in [1, s]}$  to  $\mathcal{A}_1$ .

**$H_1$  Query:**  $\mathcal{A}_1$  adaptively conducts the  $H_1$  Query for any chosen  $ID'$ .  $S$  maintains a list  $L_{H_1} = \{(ID, h_1, \tau, D)\}$  for handling the  $H_1$  Query.  $S$  checks whether  $(ID', *, *, *)$  is in  $L_{H_1}$ . If  $L_{H_1}$  contains  $ID'$ ,  $S$  retrieves the tuple  $(ID', h'_1, \tau', D')$  and returns  $D'$  to  $\mathcal{A}_1$ . Otherwise,  $S$  selects a random value  $h_1 \in Z_q^*$  and flips a coin  $\tau \in \{0, 1\}$  with a probability of  $\zeta$  when  $\tau = 1$  and  $1 - \zeta$  when  $\tau = 0$ . If  $\tau = 0$ ,  $S$  computes  $D = g^{h_1}$ . If  $\tau = 1$ ,  $S$  sets  $D = (g^\beta)^{h_1}$ .  $S$  then returns  $D$  to  $\mathcal{A}_1$  and inserts the new tuple  $(ID, h_1, \tau, D)$  into  $L_{H_1}$ .

**Partial Private Key Query:**  $\mathcal{A}_1$  adaptively executes the *PartialPrivateKeyQuery* for any selected  $ID'$ .  $S$  keeps a list  $L_{ssk} = \{(ID, sk_{ID}, x_{ID})\}$  for the *PartialPrivateKeyQuery*.  $S$  checks whether  $(ID', h'_1, \tau', D')$  is in  $L_{H_1}$ . If not,  $S$  executes the  $H_1$  Query for  $ID'$ . If  $\tau' = 1$ ,  $S$  terminates. Then,  $S$  checks whether  $(ID', *, *)$  is in  $L_{ssk}$ .

1. If  $(ID', *, *)$  is in  $L_{ssk}$ ,  $S$  checks whether  $ssk'_{ID} = \perp$ . If  $ssk'_{ID} = \perp$ ,  $S$  inquires  $(ID', h'_1, \tau', D')$  from  $L_{H_1}$ . If  $\tau' = 1$ ,  $S$  terminates. Otherwise,  $S$  computes  $ssk_{ID'} = D'^{\alpha} = g^{\alpha h'_1}$  when  $\tau' = 0$  and updates  $ssk_{ID'}$ . If  $ssk_{ID'} \neq \perp$ ,  $S$  directly retrieves  $ssk_{ID'}$  and sends it to  $\mathcal{A}_1$ .
2. If  $(ID', *, *)$  is not in  $L_{ssk}$ ,  $S$  retrieves  $(ID', h'_1, \tau', D')$  from  $L_{H_1}$ . If  $\tau' = 1$ ,  $S$  terminates. Otherwise,  $S$  computes  $ssk_{ID'} = D'^{\alpha} = g^{\alpha h'_1}$  and inserts  $(ID', sk_{ID'}, \perp)$  into  $L_{ssk}$ .

**Secret Key Query:**  $\mathcal{A}_1$  adaptively executes the *SecretKeyQuery* for any selected  $ID'$ .  $S$  checks whether  $(ID', h'_1, \tau', D')$  is in  $L_{H_1}$ . If not,  $S$  performs the  $H_1$  Query for  $ID'$ . Then,  $S$  checks whether  $(ID', *, *)$  is in  $L_{ssk}$ .

1. If  $(ID', *, *)$  is in  $L_{ssk}$ ,  $S$  checks whether  $x_{ID'} = \perp$ . If  $x_{ID'} = \perp$ ,  $S$  randomly selects  $x \in Z_q^*$  and sets  $x_{ID'} = x$ . Then,  $S$  updates  $x_{ID'}$ . If  $\tau' = 1$ ,  $S$  terminates. If  $x_{ID'} \neq \perp$ ,  $S$  retrieves  $x_{ID'}$  and sends it to  $\mathcal{A}_1$ .
2. If  $(ID', *, *)$  is not in  $L_{ssk}$ ,  $S$  randomly selects  $x \in Z_q^*$  and sets  $x_{ID'} = x$ . Then,  $S$  sends  $x_{ID'}$  to  $\mathcal{A}_1$  and inserts  $(ID', \perp, x_{ID'})$  into  $L_{ssk}$ .

**H<sub>2</sub> Query:**  $\mathcal{A}_1$  adaptively executes the  $H_2$ Query for any selected  $ID'$ .  $S$  maintains a list  $L_{H_2} = \{(ID, h_2, sk_{ID2}, pk_{ID})\}$  for the  $H_2$ Query.  $S$  checks whether  $(ID', h'_1, \tau', D')$  is in  $L_{H_1}$ . If not,  $S$  executes the  $H_1$ Query for  $ID'$ . Then,  $S$  checks whether  $(ID', *, x_{ID'})$  is in  $L_{ssk}$ . If not,  $S$  executes the *SecretKeyQuery* for  $ID'$ .  $S$  checks whether  $(ID', *, *, *)$  is in  $L_{H_2}$ .

1. If it is,  $S$  sets  $sk_{ID'2} = x_{ID'} + h'_2 \bmod q$ ,  $pk_{ID'} = g^{sk_{ID'2}}$ , then  $S$  updates  $sk_{ID'2}$  and  $pk_{ID'}$  and sends  $sk_{ID'2}$  to  $\mathcal{A}_1$ .
2. If not,  $S$  selects a random value  $h'_2 \in \mathbb{Z}_q^*$ , sets  $sk_{ID'2} = x_{ID'} + h'_2 \bmod q$ ,  $pk_{ID'} = g^{sk_{ID'2}}$ , inserts  $(ID', h'_2, sk_{ID'2}, pk_{ID'})$  into  $L_{H_2}$  and sends  $sk_{ID'2}$  to  $\mathcal{A}_1$ .

**Public Key Query:**  $\mathcal{A}_1$  adaptively executes the *PublicKeyQuery* for any selected  $ID'$ .  $S$  checks whether  $(ID', *, x_{ID'})$  is in  $L_{ssk}$ . If not,  $S$  executes the *SecretKeyQuery*. Then,  $S$  checks whether  $(ID', h'_2, sk_{ID'2}, pk_{ID'})$  is in  $L_{H_2}$ .

1. If  $(ID', h'_2, sk_{ID'2}, pk_{ID'})$  is in  $L_{H_2}$ ,  $S$  retrieves  $pk_{ID'}$  and returns it to  $\mathcal{A}_1$ .
2. If  $(ID', h'_2, sk_{ID'2}, pk_{ID'})$  is not in  $L_{H_2}$ ,  $S$  randomly selects  $h'_2 \in \mathbb{Z}_q^*$ , sets  $sk_{ID'2} = x_{ID'} + h'_2 \bmod q$ ,  $pk_{ID'} = g^{sk_{ID'2}}$ . Then,  $S$  sends  $pk_{ID'}$  to  $\mathcal{A}_1$  and inserts  $(ID', h'_2, sk_{ID'2}, pk_{ID'})$  into  $L_{H_2}$ .

**Public Key Replace:**  $\mathcal{A}_1$  adaptively runs the *PublicKeyReplace* with the selected  $(ID', PK_{ID'})$ .

1. If  $(ID', h'_2, sk_{ID'2}, pk_{ID'})$  is in  $L_{H_2}$ ,  $S$  replaces  $(ID', h'_2, sk_{ID'2}, pk_{ID'})$  in  $L_{H_2}$  with  $(ID', h'_2, \perp, pk_{ID'})$ .
2. If  $(ID', h'_2, sk_{ID'2}, pk_{ID'})$  is not in  $L_{H_2}$ ,  $S$  inserts  $(ID', \perp, \perp, pk_{ID'})$  into  $L_{H_2}$ .

**H<sub>4</sub> Query:**  $\mathcal{A}_1$  adaptively executes the  $H_4$ Query for  $H'_i$ .  $S$  maintains a list  $L_{H_4} = \{h_i, h_4, H_4\}$  for the  $H_4$ Query.  $S$  checks whether  $(h'_i, h'_4, H'_4)$  is in  $L_{H_4}$ . If it is,  $S$  retrieves  $H'_4$  and returns it to  $\mathcal{A}_1$ . Otherwise,  $S$  randomly selects a value  $h'_4 \in \mathbb{Z}_q^*$ , sets  $H'_4 = g^{h'_4}$ , inserts  $(h'_i, h'_4, H'_4)$  into  $L_{H_4}$  and returns  $H'_4$  to  $\mathcal{A}_1$ .

**Tag Query:**  $\mathcal{A}_1$  adaptively executes the *TagQuery* for  $(h_i, f_i, ID')$ .  $S$  checks whether  $\tau' = 1$  in  $L_{H_1}$  for  $ID'$ . If so,  $S$  terminates. Otherwise,  $S$  obtains  $ssk_{ID'}$  from  $L_{H_1}$ ,  $sk_{ID'2}$  from  $L_{H_2}$ , and  $H'_4$  from  $L_{H_4}$ . Then,  $S$  generates the tag for  $(h_i, f_i, ID')$  using the TagGen algorithm and sends the tag value to  $\mathcal{A}_1$ .

**Forge:**  $\mathcal{A}_1$  outputs a forged tag  $\sigma_i^*$  of block  $f_i^*$  for the identity  $ID^*$ . It is required that the block  $f_i^*$  should not have been queried in the *TagQuery* with  $ID'$ .

**Analysis:** If  $\mathcal{A}_1$  manages to win the game, the forged tag can pass the verification as follows:

$$e(\sigma_i^*, g) = e(H_1(ID^*), mpk) \cdot e\left(H_4(h_i^*) \cdot \prod_{j=1}^S A_j^{f_{ij}}, pk_{ID^*}\right).$$

According to the game rules,  $S$  extracts  $(ID^*, h_1^*, \tau^*, D^*)$  from  $L_{H_1}$  and checks the value of  $\tau^*$ . If  $\tau^* = 0$ ,  $S$  terminates. Then,  $S$  extracts  $H_1(ID^*) = g^{\beta h_1^*}$  from  $L_{H_1}$  and  $H_4(h_i^*) = g^{h_4^*}$  from  $L_{H_4}$ . Thus,  $S$  can obtain:

$$e(\sigma_i^*, g) = e(g^{\beta h_1^*}, g^\alpha) \cdot e\left(g^{h_4^*} \cdot \prod_{j=1}^S A_j^{b_{ij}}, pk_{ID^*}\right) = e(g^{\alpha \beta h_1^*}, g) \cdot e\left(pk_{ID^*}^{h_4^* + \sum_{j=1}^S a_j b_{ij}}, g\right).$$

Consequently, the result for the given CDH is  $g^{\alpha \beta} = \left(\frac{\sigma_i^*}{pk_{ID^*}^{h_4^* + \sum_{j=1}^S a_j b_{ij}}}\right)^{\frac{1}{h_1^*}}$ .

Finally, we assess the probability that  $S$  does not terminate. We denote that  $\mathcal{A}_1$  wins the game with an advantage  $\epsilon$  within time  $t$  after querying *H<sub>1</sub>Query*, *PartialPrivateKeyQuery*, *SecretKeyQuery*, *H<sub>2</sub>Query*, *PublicKeyQuery*, *PublicKeyReplace*, *H<sub>4</sub>Query* and *TagQuery* for at most  $q_{H_1}, q_{ssk}, q_{sv}, q_{H_2}, q_{pk}, q_{pkr}, q_{H_4}, q_t$  times, respectively. In the game, the probability that

$S$  does not abort is  $(1 - \zeta)^{q_{ssk} + q_t}$ . Then,  $S$  computes the value  $g^{ab}$ , that is, solves the CDH problem with at least a probability of  $\epsilon' \geq \epsilon \cdot \zeta(1 - \zeta)^{q_{ssk} + q_t} \geq \epsilon \div ((q_{ssk} + q_t) \cdot 2e)$ . The required time is  $t' \leq t + O(q_{H_1} + q_{ssk} + q_{sv} + q_{H_2} + q_{pk} + q_{pkr} + q_{H_4} + q_t)$ .  $\square$

**Theorem 2** (Unforgeability of R-tags). *If the probability of an attacker  $\mathcal{A}_2$  winning Game 2 is insignificant or negligible, then our scheme meets the requirement of R-tag unforgeability as per Definition 2.*

**Proof.** In the initial case, considering an instance  $(g, \mathbb{G}, g^\alpha, g^\beta)$ , if the probability of  $\mathcal{A}_2$  winning Game 2 is not negligible, then there exists a simulator  $S$  that can compute  $g^{\alpha\beta}$  by leveraging the capabilities of  $\mathcal{A}_2$ . In other words,  $S$  would be able to solve the Computational Diffie–Hellman (CDH) problem. The simulator  $S$  emulates the interaction with  $\mathcal{A}_2$  in the following manner.

**Setup:**  $S$  executes the Setup algorithm, generates the public parameters  $pp$ , and randomly selects  $ID_R$  as a challenge identity. Then,  $S$  randomly selects  $s$  random values  $(a_1, a_2, \dots, a_s) \in Z_q^*$  and computes  $A_k = g^{a_k}$  ( $k \in [1, s]$ ).  $S$  keeps  $msk = x$  secretly, sends  $pp$  and  $\{A_k\}_{k \in [1, s]}$  to  $\mathcal{A}_2$ .

**$H_1$  Query:**  $\mathcal{A}_2$  adaptively executes  $H_1$ Query for any selected  $ID'_O$ .  $S$  keeps a list  $L_{H_1} = \{(ID_O, h_1, \tau_1, D)\}$  for the  $H_1$ Query.  $S$  checks whether  $(ID'_O, *, *, *) \in L_{H_1}$ , if  $L_{H_1}$  contains the  $ID'_O$ ,  $S$  retrieves the tuple  $(ID', h'_1, \tau'_1, D'_1)$  and returns  $D'_1$  to  $\mathcal{A}_2$ . Otherwise,  $S$  selects a random value  $h_1 \in Z_q^*$  and flips a coin  $\tau_1 \in \{0, 1\}$  with the probability of  $\zeta$  when  $\tau_1 = 1$  and  $1 - \zeta$  when  $\tau_1 = 0$ . If  $\tau_1 = 0$ ,  $S$  computes  $D_1 = g^{h_1}$ . If  $\tau_1 = 1$ ,  $S$  sets  $D_1 = (g^\alpha)^{h_1}$ .  $S$  returns  $D_1$  to  $\mathcal{A}_2$  and inserts the new tuple  $(ID, h_1, \tau_1, D_1)$  to  $L_{H_1}$ .

**Data Owner Public Key Query:**  $\mathcal{A}_2$  adaptively executes this program for any selected data owner identity  $ID'_O$ .  $S$  keeps a list  $L_{pk_O} = \{(ID_O, h_2, D_2)\}$  for the Query.  $S$  checks whether  $(ID'_O, *, *) \in L_{pk_O}$ , if  $L_{pk_O}$  contains the  $ID'_O$ ,  $S$  retrieves the tuple  $(ID'_O, h'_2, D'_2)$  and returns  $D'_2$  to  $\mathcal{A}_2$ . Otherwise,  $S$  selects a random value  $h_2 \in Z_q^*$ ,  $S$  sets  $pk_O = ((g^\beta)^{-1})^{h_2}$ .  $S$  returns  $D$  to  $\mathcal{A}_2$  and inserts the new tuple  $(ID_O, h_2, D_2)$  to  $L_{pk_O}$ .

**Table Pro Query:**  $\mathcal{A}_2$  adaptively executes TableProQuery for any selected  $\overline{ID}_O, \overline{ID}_R$  and  $k''$ .  $S$  checks whether  $\{(ID_O, h_2, D_2)\} \in L_{pk_O}$ . If not,  $S$  performs DataOwnerPublicKeyQuery for  $ID_O$ . Otherwise,  $S$  retrieves  $D_2$  from  $L_{pk_O}$ .  $S$  keeps a list  $L_{T_{pro}} = \{(\overline{ID}_O, \overline{ID}_R, h_3, \tau_2, t)\}$  for the TableProQuery. Then,  $S$  checks whether  $(\overline{ID}_O, \overline{ID}_R, *, *, *) \in L_{T_{pro}}$ .

1. If  $(\overline{ID}_O, \overline{ID}_R, *, *, *) \in L_{T_{pro}}$ ,  $S$  retrieves  $t$ .
2. If  $(\overline{ID}_O, \overline{ID}_R, *, *, *) \notin L_{T_{pro}}$ ,  $S$  selects a random value  $h_3 \in Z_q^*$  and flips a coin  $\tau_2 \in \{0, 1\}$  with the probability of  $\zeta$  when  $\tau_2 = 1$  and  $1 - \zeta$  when  $\tau_2 = 0$ . If  $\tau_2 = 0$ ,  $S$  sets  $t = h_3$ . If  $\tau_2 = 1$ ,  $S$  computes  $t = \beta h_3$ .  $S$  inserts the new tuple  $(\overline{ID}_O, \overline{ID}_R, h_3, \tau_2, t)$  to  $L_{T_{pro}}$ .

Then,  $S$  returns  $\{D_2^t, mpk^t, k''^t\}$  to  $\mathcal{A}_2$ .

**Data Recipient Key Query:**  $\mathcal{A}_2$  handles  $pk_R$  in the same way that  $\mathcal{A}_1$  handles  $pk_O$  in the previous proof process.

**$H_4$  Query:**  $\mathcal{A}_2$  adaptively executes  $H_4$ Query for  $H_4(\overline{h}_i', \overline{W}_i')$ .  $S$  maintains a list  $L_{H_4} = \{(\overline{h}_i', \overline{W}_i'), h_4, H_4\}$  for the  $H_4$ Query.  $S$  checks whether  $(\overline{h}_i', \overline{W}_i'), h'_4, H'_4 \in L_{H_4}$ . If it does,  $S$  retrieves  $H'_4$  and returns it to  $\mathcal{A}_1$ . Otherwise,  $S$  randomly chooses a value  $h'_4 \in Z_q^*$ , sets  $H'_4 = g^{h'_4}$ , inserts  $(\overline{h}_i', \overline{W}_i'), h'_4, H'_4$  into  $L_{H_4}$  and returns  $H'_4$  to  $\mathcal{A}_2$ .

**RTag Query:**  $\mathcal{A}_2$  adaptively executes RTagQuery for  $(h_i, f_i, ID_O, ID_R)$ .  $S$  checks whether  $\tau' = 1$  in  $L_{H_1}$  for  $ID_O$ . If it does,  $S$  terminates. Otherwise,  $S$  obtains  $D_1$  from  $L_{H_1}$ ,  $D_2$  from  $L_{pk_O}$ , and  $t$  from  $L_{T_{pro}}$ . Then,  $S$  generates the R-tag for  $(h_i, f_i, ID_O, ID_R)$  by RTagGen and sends the Rtag value to  $\mathcal{A}_2$ .

**Forge:**  $\mathcal{A}_2$  outputs a forged R-tag  $\bar{\Sigma}_i$  of block  $\bar{f}_i$  for the identity  $\overline{ID}_O$  and  $\overline{ID}_R$ . It is required that the block  $\bar{f}_i$  should not be queried in  $R\text{TagQuery}$  with  $\overline{ID}_O$  and  $\overline{ID}_R$ .

**Analysis:** If  $\mathcal{A}_2$  succeeds in winning the game, the forged tag can pass the verification

$$e(\Sigma_i, g) = e\left(H_4(h'_i, \bar{W}_i) \prod_{j=1}^s A_j^{f_{ij}}, \Sigma'\right) \cdot e(H_1(ID_O), \Sigma'').$$

According to the game,  $S$  extracts  $\tau_1$  and  $\tau_2$  from  $L_{H_1}$  and  $L_{T_{pro}}$ , then checks the value of  $\tau_1$  and  $\tau_2$ . If  $\tau_1 = 0$  or  $\tau_2 = 0$ ,  $S$  terminates. Then,  $S$  extracts  $H_1(ID_O) = g^{\alpha h_1}$  from  $L_{H_1}$ ,  $\Sigma'' = g^{x\beta h_3}$  from  $L_{T_{pro}}$  and  $H_4(\bar{h}'_i, \bar{W}'_i) = g^{h_4}$  from  $L_{H_4}$ . Thus,  $S$  can obtain

$$e(\bar{\Sigma}_i, g) = e\left(g^{h_4} \cdot g^{\sum_{j=1}^s a_j f_{ij}}, \bar{\Sigma}'\right) \cdot e\left(g^{\alpha h_1}, g^{x\beta h_3}\right) = e\left(\bar{\Sigma}'^{h_4 + \sum_{j=1}^s a_j f_{ij}}, g\right) \cdot e\left(g^{\alpha\beta x h_1 h_3}, g\right).$$

Thus, the result for the given CDH is  $g^{\alpha\beta} = \left(\frac{\bar{\Sigma}_i}{\bar{\Sigma}'^{h_4 + \sum_{j=1}^s a_j f_{ij}}}\right)^{\frac{1}{x h_1 h_3}}$ .

Finally, we evaluate the probability of  $S$  not terminating. We denote that  $\mathcal{A}_1$  wins the game at the advantage  $\epsilon$  within time  $t$  after querying  $H_1\text{Query}$ ,  $\text{DataOwnerPublicKeyQuery}$ ,  $\text{TableProQuery}$ ,  $\text{DataRecipientKeyQuery}$ ,  $H_4\text{Query}$  and  $R\text{TagQuery}$  for at most  $q_{H_1}, q_{pko}, q_T, q_{pkr}, q_{pk}, q_{H_4}, q_{rt}$  times, respectively. In the game, the probability that  $S$  does not abort in the game is  $(1 - \zeta)^{q_{H_1} + q_T}$ . Then,  $S$  computes the value  $g^{ab}$ , namely, figuring out the CDH problem with at least probability of  $\epsilon' \geq \epsilon \cdot \zeta^2 (1 - \zeta)^{q_{ssk} + q_t} \geq \epsilon \setminus ((q_{ssk} + q_t) \cdot 2e)$ . The required time is  $t' \leq t + O(q_{H_1} + q_{pko} + q_T + q_{pkr} + q_{pk} + q_{H_4} + q_{rt})$ .  $\square$

**Theorem 3** (Unforgeability of Proof). *If the likelihood of an attacker ( $\mathcal{A}_3$ ) emerging victorious in game 3 is negligible in the absence of possessing the accurate data, then our scheme complies with the unforgeability of proof as per Definition 3.*

**Proof.** To make the Equation (5) hold, the cloud should generate a valid proof  $P = \{\sigma_{proof}, \{M_j\}_{j \in [1, s]}\}$  based on the undamaged data. However, the cloud must generate a proof  $P = \{\sigma'_{proof}, \{M'_j\}_{j \in [1, s]}\}$  based on the corrupted data to win game 2. If the Equation (5) is validated by using the proof, then the cloud wins the game; otherwise, they lose. Since a valid proof can pass the verification, we can obtain the following formula:

$$e(\sigma_{proof}, g) = e\left(H_1(ID_O)^{\sum_{\beta=1}^c c_\beta}, mpk\right) \cdot e\left(\prod_{\beta=1}^c H_4(h_{v_\beta}, W_{v_\beta})^{c_\beta} \cdot \prod_{j=1}^s A_j^{M_j}, pk_O\right).$$

Assuming that the CSSs emerge victorious in the game, we can also obtain the following formula:

$$e(\sigma'_{proof}, g) = e\left(H_1(ID_O)^{\sum_{\beta=1}^c c_\beta}, mpk\right) \cdot e\left(\prod_{\beta=1}^c H_4(h_{v_\beta}, W_{v_\beta})^{c_\beta} \cdot \prod_{j=1}^s A_j^{M'_j}, pk_O\right).$$

Then, we denote  $\Delta M_j = M_j - M'_j$ , where  $\sigma_{proof} = \sigma'_{proof}$ . With the properties of bilinear pairings, we derive  $\sum_{j=1}^s a_j \Delta M_j = 0$ . If we let  $G$  be a multiplicative cyclic group of a large prime  $q$ , and we suppose there are  $\lambda$  different  $\Delta M_j = M_j - M'_j$ , where  $1 \leq \lambda \leq s$ . We know that the tuples  $(a_1, a_2, \dots, a_s)$  are randomly chosen and kept secret from the cloud. Thus, the probability of making  $\sum_{j=1}^s a_j \Delta M_j = 0$  hold is less than  $q^{\lambda-1}/q^s$ , where  $q^{\lambda-1}/q^s \leq q^{\lambda-1}/q^\lambda = 1/q$ . As  $q$  is selected as a large prime,  $1/q$  is negligible. Consequently, the cloud is incapable of passing the data integrity verification by fabricating an integrity auditing proof when it does not have the correct data stored.  $\square$

### 7.3. The Detectability

**Theorem 4** (Detectability). *The proposed remote data integrity auditing scheme demonstrates a detectability level of  $\left(\frac{k}{n}, 1 - \left(\frac{n-k}{n}\right)^c\right)$ , as indicated in [35].*

**Proof.** Suppose  $Y$  represents the collection of block-signature pairs that have been tampered with by the cloud. Let  $S$  stand for the set of the block-signature pairs that are being challenged by the data owner. Denote  $X$  as the set of random variables, specifically expressed as  $X = |Y \cap S|$ . Let  $P_X$  denote the probability of identifying the corrupted data blocks. To put it another way, in the event that the cloud alters or deletes  $k$  data blocks from the original file  $F$ , the data owner is able to detect the cloud's improper behavior with probability  $P_X$  by challenging  $c$  data blocks. Hence, we obtain

$$P_X = P\{X \geq 1\} = 1 - P\{X = 0\} = 1 - \frac{n-k}{n} \times \frac{n-1-k}{n-1} \times \cdots \times \frac{n-c+1-k}{n-c+1}.$$

We can know that

$$\left(1 - \frac{n-k}{n}\right)^c \leq P_X \leq \left(1 - \frac{n-c+1-k}{n-c+1}\right)^c,$$

Given that  $\frac{n-i-1-k}{n-i-1} \leq \frac{n-i-k}{n-i}$ , it can be inferred that the proposed scheme has the ability to identify the inappropriate actions of the cloud with a probability no less than  $1 - \left(\frac{n-k}{n}\right)^c$ .  $\square$

### 7.4. Functionality Comparison

We give the functionality comparison of our scheme with several related schemes [36–38]. As shown in Table 2, our scheme is the only scheme that can satisfy all of the following properties: private verification, certificateless, data transmission.

**Table 2.** Functionality comparison with existing related schemes.

| Schemes      | Private Verification | Certificateless | Data Transmission |
|--------------|----------------------|-----------------|-------------------|
| W. Shen [36] | No                   | Yes             | No                |
| J. Shen [37] | Yes                  | Yes             | No                |
| H. Wang [38] | Yes                  | No              | Yes               |
| Our Scheme   | Yes                  | Yes             | Yes               |

### 7.5. Performance Evaluation

Our objective is to execute a comparison concerning the efficiency and functionality between our proposed plan and those of other sophisticated schemes, especially the ones proposed by W.Shen [36], J.Shen [37], and H.Wang [38]. It is important to note that when performing comparisons for multi-copy schemes, only a single copy is considered.

Table 1 shows certain definitions employed for theoretical analysis. In this table, “Pair” represents the duration required to perform a single bilinear pairing operation. “Mul” designates the time consumed in executing one multiplication operation within the group  $G_1$ , and “Exp” signifies the time necessary to conduct one exponentiation operation in  $G_1$ . Operations such as addition and multiplication in  $Z_q$ , along with hash operations, are omitted here since their associated costs are almost negligible.

During our assessment, the experimental file  $F$  is divided into  $n$  blocks. Afterwards, each block is further segmented into  $s$  sectors. The data owner (DO) chooses  $c$  blocks for challenging in each challenge. The symbol  $|p|$  stands for the size of an element within either  $G_1$  or  $G_2$ , while  $|q|$  represents the size of an element in  $Z_q$ .

Our scheme consists of only  $n$  block-tag pairs rather than  $n \times s$  block-tag pairs when the fragment structure is not utilized. In other words, as the value of  $s$  increases, the overhead associated with storing and transferring signature tags will decrease. This is because each block composed of  $s$  sectors corresponds to only one tag. Therefore, this structure has the capacity to reduce the additional storage space and communication overhead for tags, thereby enhancing the audit performance.

Moreover, the unit of measurement for communication cost is bits, and for computation cost, it is seconds. For the purpose of simplicity and to enhance readability, we will omit these units in the subsequent analysis.

### 7.5.1. Computational Overhead

An assessment of the computational cost of our proposed scheme in comparison with three other schemes is conducted by us. This evaluation focuses on the tag generation process within the setup stage and also on the proof generation as well as proof verification procedures during the proof stage, since these are recognized as the processes that consume the greatest amount of resources. A comparison among the four proposals is presented in Table 3. In this context, the variable  $c$  stands for the situation where  $c$  data blocks are being challenged by the data owner, while  $c'$  stands for the case in which  $c'$  data blocks are being challenged by the data recipient.

When it comes to the tag generation process, it can be inferred that the computation overhead of our scheme is not affected by the sector number. Moreover, it can be noticed that our scheme requires a cost of  $2mExp + 2nMul$  for computing all the tags for the DO's file.

In the proof stage, the computation overhead of the four proposals is mainly influenced by the number of the challenged blocks. In our scheme, the cloud is obliged to spend  $cExp + (c - 1)Mul$  to generate the auditing proof, and the DO needs to use up  $3Pair + (c + s + 1)Exp + (c + s)Mul$  for verifying the proof.

**Table 3.** Computational overhead comparison.

| Schemes      | TagGen                      | RTagGen             | ProofGen            | RProofGen             | ProofVerify                               | RProofCheck                                 |
|--------------|-----------------------------|---------------------|---------------------|-----------------------|-------------------------------------------|---------------------------------------------|
| W. Shen [36] | $2nExp + 2nMul$             | Invalid             | $cExp + (c - 1)Mul$ | Invalid               | $4Pair + (c + s + 3)Exp + (c + s + 2)Mul$ | Invalid                                     |
| J. Shen [37] | $2nExp + 2nMul$             | Invalid             | $cExp + (c - 1)Mul$ | Invalid               | $3Pair + (c + s + 1)Exp + (c + s - 1)Mul$ | Invalid                                     |
| H. Wang [38] | $(n(s + 1) + 1)Exp + nsExp$ | $(n + 2)Exp + nMul$ | Invalid             | $c'Exp + (c' - 1)Mul$ | Invalid                                   | $4Pair + (c' + s)Exp + (c' + s - 1)Mul$     |
| Our Scheme   | $(2n + 1)Exp + 2nMul$       | $(n + 3)Exp + nMul$ | $cExp + (c - 1)Mul$ | $c'Exp + (c' - 1)Mul$ | $3Pair + (c + s + 2)Exp + (c + s + 1)Mul$ | $7Pair + (c' + s + 1)Exp + (c' + s - 1)Mul$ |

### 7.5.2. Communication Cost

Integrity schemes involve a communication cost that is composed of multiple aspects, specifically including the transmission of a random challenge and the associated integrity proof in the proof stage. With respect to this, we present a comparison of the communication overhead, which is illustrated in Table 4.

In the proof stage, the random challenges in all four schemes have an identical size. Meanwhile, the size of the proof in our scheme reaches  $3|q|$ . In conclusion, the overall communication cost of our scheme during the proof stage is  $4|p| + s \log_2 |q|$ .

**Table 4.** Communication cost comparison.

| Schemes      | Challenge                          | Response              |
|--------------|------------------------------------|-----------------------|
| W. Shen [36] | $c \cdot ( n  +  p )$              | $ p  +  q $           |
| J. Shen [37] | $c \cdot (\log_2  n  +  q ) +  q $ | $ p  + s q $          |
| H. Wang [38] | $3 q $                             | $3 p  + s \log_2  q $ |
| Ours Scheme  | $3 q $                             | $4 p  + s \log_2  q $ |

### 7.5.3. Storage Cost

In the scheme we propose, the storage overhead is defined as the additional space required for storing particular information other than the outsourced files. The tag storage overhead, in particular, constitutes a significant part among it. When it comes to a data file that is partitioned into  $n$  blocks, our scheme has to generate  $n$  aggregation tags, and each of these tags is an element within the group  $G_1$ . Based on this, we can conclude that the storage cost for storing tags amounts to  $n|G_1|$ , which remains unaffected by the number of sectors.

We hereby demonstrate that, in comparison to the space utilized for storing the uploaded data, the storage overhead is relatively minor. For the sake of generality, we opted for a security level of 80 bits. Take a 16 MB file as an example. It can be divided into 2622 blocks, and each block can be further split into 100 sectors. We selected the elliptic curve group where  $|q| = 512$  bits. Under such conditions, our proposal entails a cost of  $2622 \times 512$  bits for storing all tags in the cloud, which is approximately equivalent to 164 KB. In this case, when we compare 164 KB with 16 MB (specifically,  $164 \text{ KB}/16 \text{ MB} \approx 1\%$ ), it is clear that the former is relatively small in proportion to the size of the outsourced file copies.

## 8. Conclusions

In this paper, we present a certificateless provable data possession scheme for cloud-based EHRs with outsourced data transmission. The scheme successfully addresses key challenges in cloud data storage and transfer. It achieves data transferability by outsourcing computations to the public cloud server, ensuring only the data owner or recipient can verify data integrity. Based on certificateless cryptography, it simplifies certificate management. Security analysis demonstrates its resistance to attacks, and performance evaluation shows its efficiency in terms of computational and communication overhead. The schemes in this paper are mainly based on traditional cryptography techniques and methods. However, the powerful computing capabilities of quantum may affect some of the encryption methods and computational efficiency therein. Therefore, in subsequent research and exploration, the performance of this scheme can be further evaluated in the field of quantum computing to determine how to adapt it to the special needs and challenges in the quantum computing environment. Meanwhile, quantum computing itself is still in the development stage, with problems such as technical limitations, errors and distortions, and scalability, which will also have an impact on the application of the scheme. Overall, the proposed scheme offers a practical and secure solution for managing EHRs in the cloud, with potential for broader application in other data-intensive fields.

**Author Contributions:** Writing—original draft and formal analysis, Y.W.; writing—review and editing, X.T.; conceptualization and supervision, Q.X. All authors have read and agreed to the published version of the manuscript.

**Funding:** This research was supported by the Hangzhou Joint Fund of the Zhejiang Provincial Natural Science Foundation of China under Grant LHZSZ24F020002 and the National Natural Science Foundation of China under Grant U21A20466.

**Data Availability Statement:** The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding authors.

**Conflicts of Interest:** The authors declare no conflicts of interest.

## References

1. Sun, J.; Fang, Y. Cross-Domain Data Sharing in Distributed Electronic Health Record Systems. *IEEE Trans. Parallel Distrib. Syst.* **2010**, *21*, 754–764. [\[CrossRef\]](#)
2. Ren, K.; Wang, C.; Wang, Q. Security Challenges for the Public Cloud. *IEEE Internet Comput.* **2012**, *16*, 69–73. [\[CrossRef\]](#)
3. Ni, J.; Zhang, K.; Yu, Y.; Yang, T. Identity-Based Provable Data Possession From RSA Assumption for Secure Cloud Storage. *IEEE Trans. Dependable Secure Comput.* **2022**, *19*, 1753–1769. [\[CrossRef\]](#)
4. Shah, P.; Prajapati, P. Provable Data Possession Using Additive Homomorphic Encryption. *J. King Saud Univ. Comput. Inf. Sci.* **2022**, *34*, 3448–3453. [\[CrossRef\]](#)

5. Guo, W.; Qin, S.; Gao, F.; Zhang, H.; Li, W.; Jin, Z.; Wen, Q. Dynamic Proof of Data Possession and Replication With Tree Sharing and Batch Verification in the Cloud. *IEEE Trans. Serv. Comput.* **2022**, *15*, 1813–1824. [\[CrossRef\]](#)
6. Li, T.; Chu, J.; Hu, L. CIA: A Collaborative Integrity Auditing Scheme for Cloud Data With Multi-Replica on Multi-Cloud Storage Providers. *IEEE Trans. Parallel Distrib. Syst.* **2023**, *34*, 154–162. [\[CrossRef\]](#)
7. Deng, L.; Wang, B.; Wang, T.; Feng, S.; Li, S. Certificateless Provable Data Possession Scheme With Provable Security in the Standard Model Suitable for Cloud Storage. *IEEE Trans. Serv. Comput.* **2023**, *16*, 3986–3998. [\[CrossRef\]](#)
8. Ateniese, G.; Burns, R.; Curtmola, R.; Herring, J.; Kissner, L.; Peterson, Z.; Song, D. Provable Data Possession at Untrusted Stores. In Proceedings of the 14th ACM Conference on Computer and Communications Security, Alexandria, VA, USA, 2 November–31 October 2007; pp. 598–609.
9. Juels, A.; Kaliski, B.S. Pors: Proofs of Retrievability for Large Files. In Proceedings of the 14th ACM Conference on Computer and Communications Security, Alexandria, VA, USA, 2 November–31 October 2007; pp. 584–597.
10. Shacham, H.; Waters, B. Compact Proofs of Retrievability. *J. Cryptol.* **2013**, *26*, 442–483. [\[CrossRef\]](#)
11. Tan, X.; Xie, Q.; Han, L.; Wang, S.; Liu, W. Proof of Retrievability with Flexible Designated Verification for Cloud Storage. *Comput. Secur.* **2023**, *135*, 103486. [\[CrossRef\]](#)
12. Wang, C.; Chow, S.S.M.; Wang, Q.; Ren, K.; Lou, W. Privacy-Preserving Public Auditing for Secure Cloud Storage. *IEEE Trans. Comput.* **2013**, *62*, 362–375. [\[CrossRef\]](#)
13. Worku, S.G.; Xu, C.; Zhao, J.; He, X. Secure and Efficient Privacy-Preserving Public Auditing Scheme for Cloud Storage. *Comput. Electr. Eng.* **2014**, *40*, 1703–1713. [\[CrossRef\]](#)
14. Guan, C.; Ren, K.; Zhang, F.; Kerschbaum, F.; Yu, J. Symmetric-Key Based Proofs of Retrievability Supporting Public Verification. In *Computer Security—ESORICS 2015*; Pernul, G., Ryan, P.Y.A., Weippl, E., Eds.; Lecture Notes in Computer Science; Springer: Cham, Switzerland, 2015; Volume 9326, pp. 203–223, ISBN 978-3-319-24173-9.
15. Shen, W.; Yu, J.; Xia, H.; Zhang, H.; Lu, X.; Hao, R. Light-Weight and Privacy-Preserving Secure Cloud Auditing Scheme for Group Users via the Third Party Medium. *J. Netw. Comput. Appl.* **2017**, *82*, 56–64. [\[CrossRef\]](#)
16. Ateniese, G.; Di Pietro, R.; Mancini, L.V.; Tsudik, G. Scalable and Efficient Provable Data Possession. In Proceedings of the 4th International Conference on Security and Privacy in Communication Networks, Istanbul, Turkey, 22–25 September 2008; pp. 1–10.
17. Erway, C.C.; K  p    , A.; Papamanthou, C.; Tamassia, R. Dynamic Provable Data Possession. *ACM Trans. Inf. Syst. Secur.* **2015**, *17*, 1–29. [\[CrossRef\]](#)
18. Wang, Q.; Wang, C.; Ren, K.; Lou, W.; Li, J. Enabling Public Auditability and Data Dynamics for Storage Security in Cloud Computing. *IEEE Trans. Parallel Distrib. Syst.* **2011**, *22*, 847–859. [\[CrossRef\]](#)
19. Yu, J.; Ren, K.; Wang, C.; Varadharajan, V. Enabling Cloud Storage Auditing with Key-Exposure Resistance. *IEEE Trans. Inform. Forensic Secur.* **2015**, *10*, 1167–1179. [\[CrossRef\]](#)
20. Yu, J.; Ren, K.; Wang, C. Enabling Cloud Storage Auditing With Verifiable Outsourcing of Key Updates. *IEEE Trans. Inform. Forensic Secur.* **2016**, *11*, 1362–1375. [\[CrossRef\]](#)
21. Yu, J.; Wang, H. Strong Key-Exposure Resilient Auditing for Secure Cloud Storage. *IEEE Trans. Inform. Forensic Secur.* **2017**, *12*, 1931–1940. [\[CrossRef\]](#)
22. Yu, J.; Hao, R.; Xia, H.; Zhang, H.; Cheng, X.; Kong, F. Intrusion-Resilient Identity-Based Signatures: Concrete Scheme in the Standard Model and Generic Construction. *Inf. Sci.* **2018**, *442–443*, 158–172. [\[CrossRef\]](#)
23. Wang, B.; Li, B.; Li, H. Oruta: Privacy-Preserving Public Auditing for Shared Data in the Cloud. *IEEE Trans. Cloud Comput.* **2014**, *2*, 43–56. [\[CrossRef\]](#)
24. Yang, G.; Yu, J.; Shen, W.; Su, Q.; Fu, Z.; Hao, R. Enabling Public Auditing for Shared Data in Cloud Storage Supporting Identity Privacy and Traceability. *J. Syst. Softw.* **2016**, *113*, 130–139. [\[CrossRef\]](#)
25. Fu, A.; Yu, S.; Zhang, Y.; Wang, H.; Huang, C. NPP: A New Privacy-Aware Public Auditing Scheme for Cloud Data Sharing with Group Users. *IEEE Trans. Big Data* **2022**, *8*, 14–24. [\[CrossRef\]](#)
26. Wang, B.; Li, B.; Li, H. Panda: Public Auditing for Shared Data with Efficient User Revocation in the Cloud. *IEEE Trans. Serv. Comput.* **2015**, *8*, 92–106. [\[CrossRef\]](#)
27. Luo, Y.; Xu, M.; Fu, S.; Wang, D.; Deng, J. Efficient Integrity Auditing for Shared Data in the Cloud with Secure User Revocation. In Proceedings of the 2015 IEEE Trustcom/BigDataSE/ISPA, Helsinki, Finland, 20–22 August 2015; pp. 434–442.
28. Wang, H. Identity-Based Distributed Provable Data Possession in Multicloud Storage. *IEEE Trans. Serv. Comput.* **2015**, *8*, 328–340. [\[CrossRef\]](#)
29. Wang, H.; He, D.; Tang, S. Identity-Based Proxy-Oriented Data Uploading and Remote Data Integrity Checking in Public Cloud. *IEEE Trans. Inform. Forensic Secur.* **2016**, *11*, 1165–1176. [\[CrossRef\]](#)
30. Yu, Y.; Au, M.H.; Ateniese, G.; Huang, X.; Susilo, W.; Dai, Y.; Min, G. Identity-Based Remote Data Integrity Checking With Perfect Data Privacy Preserving for Cloud Storage. *IEEE Trans. Inform. Forensic Secur.* **2017**, *12*, 767–778. [\[CrossRef\]](#)
31. Wang, H.; He, D.; Yu, J.; Wang, Z. Incentive and Unconditionally Anonymous Identity-Based Public Provable Data Possession. *IEEE Trans. Serv. Comput.* **2019**, *12*, 824–835. [\[CrossRef\]](#)
32. Zhang, Y.; Yu, J.; Hao, R.; Wang, C.; Ren, K. Enabling Efficient User Revocation in Identity-Based Cloud Storage Auditing for Shared Big Data. *IEEE Trans. Dependable Secure Comput.* **2018**, *17*, 608–619. [\[CrossRef\]](#)
33. Shen, W.; Yang, G.; Yu, J.; Zhang, H.; Kong, F.; Hao, R. Remote Data Possession Checking with Privacy-Preserving Authenticators for Cloud Storage. *Future Gener. Comput. Syst.* **2017**, *76*, 136–145. [\[CrossRef\]](#)

34. Ma, H.; Tian, G.; Liu, Z.; Zhang, L. Secure Data Deduplication with Ownership Management and Sharing in Cloud Storage. In *Frontiers in Cyber Security*; Li, F., Takagi, T., Xu, C., Zhang, X., Eds.; Communications in Computer and Information Science; Springer: Singapore, 2018; Volume 879, pp. 168–176, ISBN 9789811330940.
35. Miao, Y.; Huang, Q.; Xiao, M.; Susilo, W. Blockchain Assisted Multi-Copy Provable Data Possession With Faults Localization in Multi-Cloud Storage. *IEEE Trans. Inform. Forensic Secur.* **2022**, *17*, 3663–3676. [[CrossRef](#)]
36. Shen, W.; Qin, J.; Yu, J.; Hao, R.; Hu, J. Enabling Identity-Based Integrity Auditing and Data Sharing With Sensitive Information Hiding for Secure Cloud Storage. *IEEE Trans. Inform. Forensic Secur.* **2019**, *14*, 331–346. [[CrossRef](#)]
37. Shen, J.; Zeng, P.; Choo, K.-K.R.; Li, C. A Certificateless Provable Data Possession Scheme for Cloud-Based EHRs. *IEEE Trans. Inform. Forensic Secur.* **2023**, *18*, 1156–1168. [[CrossRef](#)]
38. Wang, H.; He, D.; Fu, A.; Li, Q.; Wang, Q. Provable Data Possession with Outsourced Data Transfer. *IEEE Trans. Serv. Comput.* **2021**, *14*, 1929–1939. [[CrossRef](#)]

**Disclaimer/Publisher’s Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.